

Aarya Vijay Arban
Batch – S11
Roll no. – 07

Aim – Implementation of evaluation of postfix expression

Code:

```
#include <stdio.h>

#include <stdlib.h>

#define MAX_SIZE 100

// Stack implementation

int stack[MAX_SIZE];

int top = -1;

void push(int item) {
    if (top >= MAX_SIZE - 1) {
        printf("Stack Overflow\n");
        return;
    }
    top++;
    stack[top] = item;
}

int pop() {
    if (top < 0) {
        printf("Stack Underflow\n");
        return -1;
    }
    int item = stack[top];
    top--;
    return item;
}
```

```

int is_operator(char symbol) {
    if (symbol == '+' || symbol == '-' || symbol == '*' || symbol == '/') {
        return 1;
    }
    return 0;
}

int evaluate(char* expression) {
    int i = 0;
    char symbol = expression[i];
    int operand1, operand2, result;

    while (symbol != '\0') {
        if (symbol >= '0' && symbol <= '9') {
            int num = symbol - '0';
            push(num);
        }
        else if (is_operator(symbol)) {
            operand2 = pop();
            operand1 = pop();
            switch(symbol) {
                case '+': result = operand1 + operand2; break;
                case '-': result = operand1 - operand2; break;
                case '*': result = operand1 * operand2; break;
                case '/': result = operand1 / operand2; break;
            }
            push(result);
        }
        i++;
        symbol = expression[i];
    }
}

```

```

    }

    result = pop();

    return result;
}

int main() {
    char expression[] = "5 6 7 + * 8 -";

    int result = evaluate(expression);

    printf("Result= %d\n", result);

    return 0;
}

```

Output:

The output of the above program will be:

Result: 57

Algorithm:

1. Start scanning the expression from **left to right**.
2. Push operand **5** onto the stack.
3. Push operand **6** onto the stack.
4. Push operand **7** onto the stack.
5. Pop operands **7** and **6** from the stack, perform addition, and push the result **(13)** back onto the stack.
6. **Pop operands 13 and 5** from the stack, perform multiplication, and **push the result (65)** back onto the stack.
7. Push **operand 8** onto the stack.
8. **Pop operands 8 and 65** from the stack, perform subtraction, and push the result **(57)** back onto the stack.
9. The final result is **57**.